

React Learning Tree

Complete roadmap of React concepts used in companies

core — Must-know basics

important — Heavily used in companies

advanced — Senior-level concepts

1. Core Fundamentals

JSX & Rendering

core

- What is JSX and how it compiles
- Expressions inside JSX { }
- Conditional rendering (&&, ternary)
- Lists & keys — why keys matter
- Fragments <>...
- Self-closing tags & className vs class

Components

core

- Functional components (modern standard)
- Class components (legacy — still in companies)
- Props & prop types
- Default props
- Children prop
- Pure components — same input = same output

State & Events

core

- useState — state declaration & updates
- State is immutable — never mutate directly
- Batched state updates
- Event handling — onClick, onChange, onSubmit
- Synthetic events in React
- Controlled vs uncontrolled inputs

2. Hooks (Most Critical)

Basic Hooks

core

- useState — full deep dive
- useEffect — side effects & lifecycle
- useEffect cleanup function
- Dependency array — [], [dep], no array
- useRef — DOM refs & mutable values
- useContext — consuming context

Performance Hooks

important

- useMemo — memoize expensive calculations
- useCallback — memoize functions
- When to use vs not use memo/callback
- React.memo — prevent re-renders

Advanced Hooks

advanced

- `useReducer` — complex state logic
- `useLayoutEffect` — synchronous DOM updates
- `useImperativeHandle` + `forwardRef`
- `useId` — unique IDs for accessibility
- `useDeferredValue` — low-priority updates
- `useTransition` — non-blocking UI

Custom Hooks

important

- Writing your own hooks — `useFetch`, `useDebounce` etc.
- Rules of hooks
- Extracting logic into reusable hooks
- Custom hooks in company codebases

3. Component Patterns

Composition Patterns

important

- Component composition vs inheritance
- Compound components pattern
- Render props pattern
- Higher-Order Components (HOC)
- Slot pattern with children
- Container vs presentational components

State Patterns

important

- Lifting state up
- State co-location principle
- Controlled components pattern
- Uncontrolled + `useRef` pattern
- State initialization patterns

Advanced Patterns

advanced

- Provider pattern with `Context`
- Observer pattern
- Factory pattern for components
- Polymorphic components (as prop)
- Headless components

4. State Management

React Context API

core

- `createContext`, `Provider`, `Consumer`
- `useContext` hook usage
- Context performance pitfalls
- Multiple contexts pattern
- When to use context vs props

Redux Toolkit (RTK)

important

- `configureStore`, `createSlice`, `createReducer`
- Actions & reducers

- useSelector & useDispatch
- RTK Query — data fetching built-in
- Redux DevTools
- Middleware — Thunk, Saga basics

Zustand & Others

advanced

- Zustand — simple global state
- Jotai — atomic state
- Recoil — Facebook's state library
- When to choose what

5. Data Fetching

Fetching Basics

core

- fetch API inside useEffect
- Axios — most used in companies
- Loading & error states
- Abort controller — cancel requests
- useEffect cleanup for fetch

TanStack Query (React Query)

important

- useQuery — fetch & cache data
- useMutation — POST/PUT/DELETE
- Query invalidation & refetching
- Stale time & cache time
- Infinite queries — pagination
- Optimistic updates

SWR

advanced

- useSWR — Vercel's fetching library
- Revalidation strategies
- SWR vs React Query comparison

6. Routing

React Router v6

core

- BrowserRouter, Routes, Route
- Link & NavLink
- useNavigate, useParams, useLocation
- Nested routes & Outlet
- Index routes
- Protected / private routes
- 404 page — catch-all route
- Lazy loading routes

Advanced Routing

advanced

- Search params — useSearchParams
- Route loaders & actions (v6.4+)
- React Router + Redux pattern

7. Forms

Controlled Forms

core

- Controlled inputs with useState
- Form submission handling
- Validation logic — manual approach
- Multiple field forms

Form Libraries

important

- React Hook Form — most popular in companies
- register, handleSubmit, errors
- Yup / Zod — schema validation
- Formik — older but still used
- Field arrays — dynamic form fields

8. Styling

CSS Approaches

core

- Plain CSS & CSS Modules (scoped styles)
- Inline styles — when & how
- BEM naming convention

CSS-in-JS & Utility

important

- Tailwind CSS — most popular in companies
- Styled Components
- Emotion — CSS-in-JS
- Sass/SCSS modules
- Conditional classNames — clsx library

UI Libraries

important

- Material UI (MUI)
- Shadcn/ui — copy-paste components
- Ant Design
- Chakra UI
- Radix UI — headless primitives

9. Performance

Re-render Optimization

important

- React.memo — skip re-renders
- useCallback & useMemo correctly
- Avoiding unnecessary state
- Key prop usage & issues
- Why components re-render — mental model

Code Splitting

important

- React.lazy & Suspense
- Dynamic imports
- Route-based splitting
- Component-level splitting

Advanced Performance

advanced

- Virtualization — react-window, react-virtual
- Concurrent features
- useTransition & useDeferredValue
- React Profiler in DevTools
- Bundle size analysis — webpack-bundle-analyzer

10. Testing

Unit & Component Testing

important

- Jest — test runner
- React Testing Library — most used
- render, screen, fireEvent, userEvent
- getBy, findBy, queryBy queries
- Mocking modules & API calls

E2E Testing

advanced

- Cypress — browser-level testing
- Playwright — modern alternative
- Writing user flow tests

11. Advanced React

Error Handling

important

- Error Boundaries — class component requirement
- Catching render errors
- Fallback UI patterns
- react-error-boundary library

Portals & Refs

important

- ReactDOM.createPortal — modals, tooltips
- forwardRef — ref forwarding
- useImperativeHandle
- Callback refs

React Internals

advanced

- Virtual DOM & reconciliation
- Fiber architecture basics
- Diffing algorithm
- Concurrent Mode concepts
- Server Components (RSC) — React 18+

React 18+ Features

advanced

- Automatic batching
- Concurrent rendering
- Suspense improvements
- useId, useTransition, useDeferredValue
- createRoot API

12. Ecosystem & Tooling

Project Setup

core

- Vite — modern, fast (preferred over CRA)
- Create React App (legacy)
- ESLint + Prettier config
- Absolute imports setup
- Environment variables (.env)

TypeScript in React

important

- Typing props with interfaces
- Typing useState, useRef, useContext
- Generic components
- Event types
- FC type vs function declaration

Frameworks

advanced

- Next.js — SSR, SSG, App Router
- Remix — full-stack React
- Gatsby — static sites
- When to use a framework vs plain React

DevTools & Debugging

important

- React DevTools extension
- Profiler tab usage
- Strict Mode & its double-invocation
- Why Did You Render library